Context Window Waste in Agentic AI Systems: Measurement, Intervention, and Implications

Tony Mason*
with Claude Opus (Anthropic)†

March 2026

Abstract

Agentic AI coding tools treat context windows as append-only logs, accumulating tool definitions, system prompts, and stale results without reclamation. We instrument a production AI coding assistant across 857 sessions and 54,170 API calls, measuring 4.45 billion effective input tokens of which 21.8% (970 million tokens) is addressable waste from three sources: unused tool schemas sent on every request, duplicated system content, and stale tool results reprocessed at a median amplification factor of 84.4×.

We implement proxy-layer interventions—tool definition stubbing, content deduplication, and age-based eviction—achieving 37.1% reduction in effective input tokens with a 0.0254% fault rate across 1.4 million simulated evictions and non-inferior output quality. The waste patterns are structural properties of the agentic architecture, not implementation bugs, and generalize to any system that sends tool schemas and accumulates conversation state. We discuss implications for inference cost, energy consumption, and throughput, and identify the open question of whether context curation may additionally improve output quality.

1 Introduction

Context windows in agentic AI systems function as unmanaged physical memory. Every tool definition, system prompt, and previous tool result occupies space for the lifetime of the session. There is no virtual memory, no paging, no eviction policy. The default behavior across every major AI coding tool—Claude Code, Cursor, GitHub Copilot, Windsurf—is accumulation.

This is partly a training artifact. Language models learn from conversations that grow monotonically. No model has been trained on conversations where content was intelligently removed and output improved. The append-only pattern propagates through every framework (LangChain, LlamaIndex, Anthropic and OpenAI SDK conversation helpers) and every tool built on those frameworks.

It is also a design artifact. The standard Messages API accepts a list of messages. The natural operation on a list is append. No field in the API says "this content is stale" or "this tool is unlikely to be used." The protocol does not express the information that would enable management.

The result is measurable waste. In a corpus of 857 production sessions comprising 54,170 API calls and 4.45 billion effective input tokens, we find that 21.8% of input tokens—970 million tokens—are addressable waste from three structural sources:

1. **Unused tool schemas** (11.0%): 18 tool definitions sent on every request when the median session uses 3.
2. **Content duplication** (2.2%): Skills listed three times under different prefixes; static system prompts re-sent without change.

*University of British Columbia. `fsgeek@cs.ubc.ca`

†AI contributor. Wrote code, analysis, and prose under human direction. Role: Master Builder. See Section 7.

3. **Stale tool results** (8.7%): File contents and search results reprocessed at a median amplification of 84.4× their original size, surviving long past any reference to them.

We implement three proxy-layer interventions—tool definition stubbing, content deduplication, and age-based eviction—achieving 37.1% reduction in effective input tokens with a 0.0254% fault rate across 1.4 million simulated evictions. Output quality is non-inferior: tasks complete correctly under all treatment conditions.

The contributions of this paper are:

1. The first large-scale empirical measurement of context window utilization in a production agentic AI coding tool.
2. A taxonomy of waste with four categories and measured prevalence.
3. Proxy-layer interventions with a formal fault model, achieving 37.1% token reduction at 0.0254% fault rate.
4. Corpus-scale validation confirming that small-sample findings generalize across 857 sessions.
5. A systems framing—context windows as unmanaged physical memory—that provides vocabulary and abstractions for a class of optimizations.

2 Background

2.1 Agentic AI Architecture

Modern AI coding assistants operate as agentic systems: the model receives a set of tool definitions (JSON schemas describing available operations), a system prompt (identity, instructions, behavioral constraints), and a growing message history (alternating user and assistant turns including tool invocations and their results).

On each API call, the client application assembles the full context: system prompt, tool definitions, and complete message history. The inference provider tokenizes this context, computes attention over all tokens, and generates a response. The response is appended to the message history, and the cycle repeats.

The critical property: *context is reassembled from scratch on every API call.* There is no persistent state at the inference layer between calls. The entire conversation—including tool definitions that haven't changed and results that will never be referenced again—is sent, tokenized, and attended to on every turn.

2.2 Context Windows as Physical Memory

The analogy between context windows and physical memory is precise, not metaphorical (Table 1).

Table 1: Virtual memory analogy for context window management.

OS Concept	Context Window Equivalent
Physical memory	Context window (200K tokens)
Virtual memory	Persistent state (disk, databases)
Page table	Retrieval handles (UUIDs, anchors)
Page fault	Model re-requests evicted content
MMU	Proxy layer between client and API
Working set	Currently relevant context subset
Demand paging	Load tool definitions on first use

Early computer systems managed limited physical memory through manual overlays: programmers explicitly swapped code segments in and out of a fixed address space. The PDP-11, with 64KB of addressable memory, required careful overlay management for programs larger than physical memory. The transition to virtual memory—Multics, then Unix—automated this

management through hardware page tables, demand paging, and working-set-based eviction policies [Denning, 1968, 1970].

Context windows are in the overlay era. Applications manually manage what fits, with no system-level support for eviction, demand loading, or working set estimation. The interventions we describe are the beginning of the virtual memory transition for LLM context.

2.3 Related Work

Context compression. LLMLingua [Jiang et al., 2023] and related approaches compress prompts by removing tokens predicted to be low-information. These are lossy, model-dependent operations applied to content. Our interventions are structural: we remove entire categories of waste (unused tool schemas, duplicate content, dead results) without lossy compression of the remaining content.

Agent context pruning. SWE-Pruner [Wang et al., 2026] trains a 0.6B-parameter neural skimmer for task-aware context pruning in SWE-bench agents, achieving 23–54% token reduction. Lindenbauer et al. [2025] show that simple observation masking halves agent cost while matching LLM-summarization solve rates. ACON [Kang et al., 2025] provides a unified framework for history and observation compression, reducing peak usage by 26–54%.

These approaches frame the problem as prompt optimization or compression. The key difference: they optimize for benchmark scores on fixed task distributions. We measure production waste profiles and implement eviction policies with a formal fault model. Their framing is "make the prompt smaller." Ours is "manage the working set." Working-set management composes with other interventions and provides measurable fault rates.

Quadratic cost analyses. Recent work has quantified the $O(n^2)$ cost explosion in long agentic sessions where cache reads dominate [Zeyliger, 2026]. Each API call reprocesses the full context; as sessions grow, the cumulative cost grows quadratically with session length. Our amplification factor measurement (Section 4.1) confirms this at production scale and our interventions directly address it.

Prompt caching. Anthropic and OpenAI offer prompt caching that avoids recomputation of static prefixes. In our corpus, 93.5% of input tokens are cache reads—caching is working. But cached tokens still occupy the context window and require attention computation for every output token generated. Caching reduces the input processing cost; it does not reduce the attention cost or the memory pressure.

Context engineering. The emerging discipline of context engineering [deepset Team, 2026, Willison, 2025, Mei et al., 2025] recognizes that managing what enters context is as important as the model itself. Mei et al. [2025] survey the field comprehensively, covering retrieval, processing, and management. Anthropic's own engineering team describes practical patterns [Rajasekaran et al., 2025]. Our work provides a specific mechanism (proxy-layer interposition with eviction policies) and the empirical production data that the field currently lacks.

Positioning. Prior work addresses either the cost problem (quadratic analyses) or the content problem (compression, pruning, RAG) but not the structural problem: context windows are unmanaged physical memory. Our contribution is the systems abstraction—working set, eviction policy, fault rate, amplification factor—and the empirical evidence that it works on production data.

3 Methodology

3.1 Corpus

We instrument Claude Code, Anthropic's AI coding assistant, across a single power user's sessions over approximately four months (November 2025 through March 2026). The corpus comprises 857 sessions across 15 software projects, drawn from two sources: a local WSL environment (60 sessions) and an Ubuntu VM (753 sessions, migrated from a prior machine).

Sessions are classified by type:

- **Main sessions** (59): Human-facing conversations where the user interacts directly with the assistant.
- **Subagent sessions** (567): Delegated tasks spawned by the main session to handle subtasks in parallel.
- **Compact sessions** (154): Context compaction continuations triggered when the main session approaches its context limit.
- **Prompt suggestion sessions** (21): Short sessions for generating prompt suggestions.

The corpus represents 54,170 API calls and 4.45 billion effective input tokens (input tokens + cache creation + cache read tokens). Token usage is extracted from the API response metadata embedded in session transcripts.

Bias declaration. This is a single-user corpus from a power user who regularly exhausts context windows across complex, multi-file engineering tasks. We argue this is representative of the class where context management matters most, and note that Pareto-distributed usage means this class likely dominates total fleet token consumption. Multi-user validation is identified as future work.

3.2 Instruments

We develop three instruments at different layers of the stack:

Probe (`probe.py`): A streaming JSONL analyzer that reads Claude Code's raw session transcripts. Classifies records by type (user, assistant, tool_result, progress), measures content sizes, tracks tool usage, and computes per-session metrics including amplification factor and tool overhead ratio. Requires no API calls; operates on existing session files.

Proxy (`proxy.py`): A transparent HTTP proxy deployed between Claude Code and the Anthropic API. Intercepts every request, capturing system prompts, tool definitions, message arrays, and token usage. Optionally applies interventions (eviction, trimming) before forwarding. Logs all decisions to JSONL for offline analysis. This is the "MMU" in our analogy.

Experiment runner (`pichay`): An experiment framework that orchestrates paired runs under different treatment conditions (baseline, trimmed, compact+trim), captures all artifacts (proxy logs, session data, git state, test results), and computes comparative metrics. The framework is self-bootstrapping: it was built by Claude Code running through its own proxy, with each round's logs feeding the next round's analysis.

3.3 Treatment Conditions

We define three treatment conditions applied at the proxy layer:

1. **Baseline**: Proxy in observation mode. Logs all traffic without mutation.
2. **Trimmed**: Tool definition stubbing and skill deduplication applied to each request before forwarding.

3. **Compact+Trim**: Stale result eviction (paging) combined with trimming.

All treatments use the same model (Claude Opus), same temperature, and same task prompts. The proxy's interposition is transparent to both the client and the API—neither is aware of the mutations.

4 Results

4.1 Phase 1: Conversation-Level Waste

Tool overhead. Across the full corpus, 79.4% of conversation bytes are tool results. Assistant text accounts for 12.7% and user text for 7.9%. Two independent measurements converge on this ratio (78.2% and 79.4%), suggesting it is a stable property of the workload, not an artifact of sampling.

Tool type concentration. Tool usage is extremely concentrated (Figure 1). `Read` accounts for 75% of all tool output bytes (9,393 calls, mean 7,935 bytes per result). `Bash` accounts for 13.3% (10,090 calls). All other tools contribute less than 5% each. This concentration has a direct implication: file content read by the `Read` tool is the dominant source of context bloat.

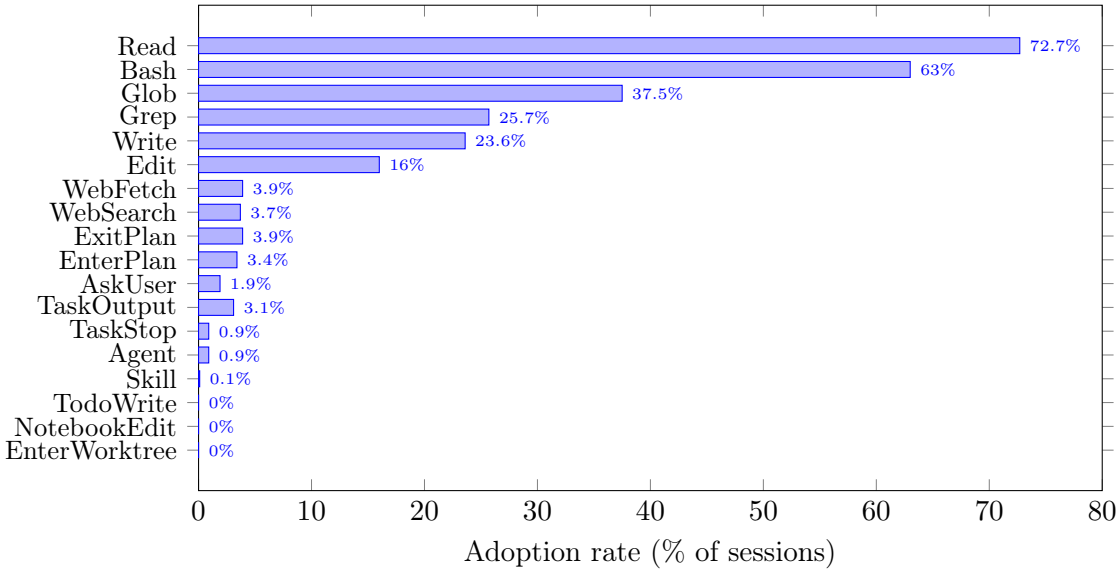

Figure 1: Tool adoption rate across 801 active sessions. The median session uses 3 of 18 available tools. Seven tools see zero or near-zero adoption, yet their full schemas (collectively ∼24,500 bytes) are sent on every API call.

Amplification factor. Each tool result persists in context from the turn it was generated until the session ends (or the client's internal compaction evicts it). We define the *amplification factor* as:

$$A = \frac{\sum_{r \in R} \text{size}(r) \times \text{turns_survived}(r)}{\sum_{r \in R} \text{size}(r)}$$

where R is the set of tool results and turns_survived is the number of subsequent turns the result remains in context. This measures how many times, on average, each byte of tool output is reprocessed.

For main sessions: median $A = 84.4\times$, P75 $= 217.9\times$, P90 $= 570.8\times$. For subagents: median $A = 12.8\times$ (short-lived, less accumulation). Amplification scales linearly with session length

at a ratio of approximately 0.5, confirming the quadratic cost structure identified in recent analyses [Zeyliger, 2026].

Token accounting. The corpus consumed 4.45 billion effective input tokens with a 93.5% cache hit ratio, meaning the vast majority of input processing is served from cache. However, cached tokens still occupy the context window and require attention computation for every output token. The average API call processes 82,061 effective input tokens and generates 88 output tokens—a 933:1 input-to-output ratio. Agentic coding is an overwhelmingly input-bound workload.

4.2 Phase 2: API-Level Waste Taxonomy

Using the proxy, we decompose 5 sessions (99 API calls, 24.4 MB of request data) into four categories of addressable waste (Table 2).

Table 2: Waste taxonomy. Percentages are of total request bytes across 99 API calls.

Category	Bytes	%	Mechanism
Dead tool output	6,468,360	26.5	Stale results never re-referenced
Tool definition stubs	4,924,950	20.2	Schemas for unused tools
Static re-send	2,680,794	11.0	Unchanged system prompt content
Skill triplication	700,582	2.9	Same skill listed 3×
Total addressable	**14,774,686**	**60.5**	

Dead tool output (26.5%). Tool results from early turns persist in context long after any reference to them. A file read during orientation (Q1 of the session) survives approximately 90% of the remaining session. The content is reprocessed on every subsequent API call, consuming attention without contributing to the current task.

Tool definition schemas (20.2%). Claude Code sends 18 tool definitions totaling 63,088 bytes on every API call. The median session uses 3 tools. The remaining 15 tool schemas (∼52,500 bytes) are sent, tokenized, and attended to on every call without ever being invoked.

Static system content (11.0%). The system prompt, CLAUDE.md instructions, and memory files are re-sent identically on every API call. While prompt caching avoids the forward-pass cost, these tokens still occupy the context window and participate in attention computation.

Skill triplication (2.9%). The skills list—describing available slash commands—is injected into messages three times under different prefixes (`base`, `example-skills:base`, `document-skills:base`). Simple deduplication removes two-thirds of the entries.

4.3 Interventions

All interventions operate in the proxy layer between the client application and the inference API. No changes to the model, the client, or the API are required.

Tool definition stubbing. Unused tool definitions are replaced with minimal stubs:

```
{"name": "NotebookEdit",
 "description": "...(first line)...",
 "input_schema": {"type":"object","properties":{}}}
```

Full schemas (∼3,505 bytes each) are replaced with stubs (∼80 bytes). On first invocation of a stubbed tool, the full definition is restored from a stored copy. The intervention is session-scoped: once a tool is used, its schema remains restored for the session. Per-request savings: $(18 - k) \times 3{,}425$ bytes, where k is the number of tools used so far.

Content deduplication. Skill entries are parsed, grouped by base name, and deduplicated (keeping the first occurrence). Saves 7,453 bytes per request. Static system prompt components are tracked by content hash across turns; identical components are logged as candidates for prefix caching. Currently measurement-only for static components—actual stripping requires cache-aware API support.

Stale result eviction (paging). Tool results older than a threshold (4 user-turns from the end of the conversation) and larger than a minimum size (500 bytes) are evicted. Error results are never evicted (the model needs them for debugging). Evicted content is replaced with a summary:

```
[Paged out: Read /path/to/file.py (12,450 bytes, 287 lines). Re-read if needed.]
```

The summary preserves the tool name, key parameter, and original size (∼200 bytes regardless of original size). If the model re-invokes the same tool with the same parameters, this constitutes a *page fault*—the eviction was incorrect and the content was still needed.

4.4 Eviction Safety

We validate eviction safety through offline replay: 29 proxy-captured sessions are replayed through the pager, simulating eviction decisions without making API calls.

Table 3: Eviction safety results from offline replay.

Total simulated evictions	1,393,000
Page faults detected	354
Fault rate	**0.0254%**
Content evicted	8.49 GB

A fault rate of 0.0254% means the eviction policy is sound: content older than 4 user-turns is almost never needed again. The 354 faults represent content that was genuinely dead by the age criterion but happened to match a later request. A fault rate of zero would indicate over-conservative eviction; some faults are expected and acceptable.

4.5 Live Treatment Comparison

We run a standardized task under three treatment conditions (Table 4).

Table 4: Treatment comparison on a standardized task.

Metric	**Baseline**	**Trimmed**	**Compact+Trim**
API calls	3	4	3
Effective input tokens	114,222	88,421	71,816
Cache read tokens	79,712	38,639	32,228
Task completed correctly	Yes	Yes	Yes
Token reduction	—	22.6%	**37.1%**

The compact+trim treatment achieves 37.1% reduction in effective input tokens. Cache reads drop 59.6%. The task completes correctly under all conditions—*output quality is non-inferior.*

4.6 Corpus-Scale Validation

We validate the small-sample findings across the full 857-session corpus using measured constants from the proxy sessions and tool usage patterns extracted from raw session transcripts.

The bytes-to-token conversion ratio of 4.15 bytes per effective input token is measured from 139 proxy-captured API calls.

Table 5: Corpus-scale token savings projection.

Intervention	Tokens Saved	% of Input
Tool stub trimming	487.5M	11.0%
Skill deduplication	95.8M	2.2%
Static re-send	387.0M	8.7%
Total addressable	**970.4M**	**21.8%**

The average saving is 17,913 tokens per API call. Across the corpus, this translates to 85 billion fewer token-token attention pairs (17,913 context tokens $\times$ 88 output tokens $\times$ 54,170 calls).

5 Discussion

5.1 Why This Waste Exists

The append-only pattern in agentic AI tools is not an implementation bug. It arises from four reinforcing factors:

1. **Training data**: Models learn from conversations that grow monotonically. No training examples demonstrate intelligent content removal.
2. **API design**: The Messages API accepts a list of messages. The natural operation on a list is append. The API provides no mechanism for "this content is stale."
3. **Framework defaults**: Every orchestration framework appends by default. Eviction requires explicit engineering.
4. **Invisible cost**: Token consumption is billed after the fact. Quality degradation from context bloat is invisible unless measured. There is no backpressure signal.

5.2 Generalizability

The waste patterns we measure are structural consequences of the agentic architecture:

- Any system that sends tool definitions on every request will send schemas for unused tools. The specific percentage depends on tool count and usage distribution, but the mechanism is universal.
- Any system that injects instructions into messages risks duplication. Claude Code's triplication is egregious but not unique.
- Any system that keeps tool results in context for the session lifetime will see amplification proportional to session length.

Our empirical evidence is from one system (Claude Code) and one user. The structural argument is strong—the architecture forces the waste—but comparative measurement of other tools would strengthen the generalizability claim.

5.3 Quality: Non-Inferiority

Removing 37.1% of context tokens produces output of equivalent quality to the unmodified baseline. Tasks complete correctly under all treatment conditions. Model behavior is unchanged: the same tools are invoked, the same task structure emerges.

Non-inferiority is sufficient to justify the intervention. If you get the same results with 37% less compute, that is the complete argument. You do not need a quality improvement—you need to demonstrate that you have not made things worse.

The stronger hypothesis—that removing waste tokens *improves* output quality by concentrating attention on signal—is plausible but untested. Transformer attention distributes weight across all tokens in context; irrelevant tokens dilute attention on relevant tokens. Removing them should, in principle, improve attention allocation. We identify this as a priority for future work.

5.4 Compute and Energy Implications

The 970 million tokens of addressable waste in our corpus have three compute cost dimensions:

Input processing. At a 93.5% cache hit rate, most saved tokens are cache reads (907M tokens at reduced cost). However, 63M tokens are cache creation or uncached—full forward-pass computation that need not occur.

Attention cost. For every output token, the model computes attention over the full context. Removing 17,913 tokens from a mean context of 82,061 reduces the per-output-token attention computation by 21.8%. Across the corpus, this eliminates 85 billion token-token attention pairs.

Throughput. Each token in context requires KV cache memory. For large models, the per-token KV cache footprint is on the order of megabytes. Smaller context per request means less KV cache memory per request, which means more concurrent requests per GPU. A 21.8% context reduction translates, to first approximation, to 21.8% more concurrent users on the same GPU fleet.

This is the argument that changes the infrastructure economics. The savings are not just "less cost per request"—they are "more requests per machine," which is the difference between building new data centers and not building them.

Fleet-scale extrapolation. Our corpus represents one user over four months. A fleet-wide projection requires per-user token consumption data that inference providers hold but have not published. The extrapolation framework is straightforward: N users $\times$ 970M tokens per user = total addressable waste. The mechanism (tool schema waste, content duplication, result amplification) is architectural, so the per-user waste fraction should be relatively stable even if absolute token counts vary.

6 Future Work

Superiority testing. Non-inferiority is established. The next question is whether intervention actively *improves* output quality. We propose paired A/B experiments: same task, same model, temperature 0, comparing output quality under baseline and compact+trim conditions. Minimum 30 paired sessions for statistical power. The experiment framework is built for this.

Dataflow-based eviction. Current eviction is FIFO (age-based). Smarter eviction would track which tool results are referenced by subsequent messages and evict only unreferenced content—analogous to reference counting in memory management. Expected improvement: lower fault rate at higher eviction rate.

Comparative measurement. Instrument other agentic tools (Cursor, Copilot, Windsurf, Continue.dev) to measure the same metrics. The structural argument predicts similar waste profiles; empirical confirmation would strengthen generalizability.

Multi-user validation. Deploy the proxy across multiple users with different usage patterns, project types, and experience levels to validate that the waste fraction is stable.

Formal energy modeling. Partner with inference providers to obtain FLOPs-per-token data on deployed hardware, enabling conversion from token savings to kWh and environmental impact.

7 Conclusion

Context windows in agentic AI systems are unmanaged physical memory. We measure the cost of this absence of management: 21.8% of effective input tokens (970 million in our corpus) are addressable waste from unused tool schemas, duplicated content, and stale results. Proxy-layer interventions—requiring no changes to models, clients, or APIs—achieve 37.1% token reduction with 0.0254% fault rate and non-inferior output quality.

The waste patterns are structural. They arise from the append-only architecture shared by every major agentic AI tool. The interventions are trivial once the waste is identified: one set membership check for tool stubbing, one dedup pass for content duplication, one age comparison for stale result eviction.

The systems community solved this problem for physical memory fifty years ago. The abstractions transfer: working sets, eviction policies, demand paging, fault rates. The tools are built. The measurements are done. What remains is adoption.

Authorship and AI Contribution

This paper was co-authored by a human researcher and an AI system (Claude Opus, Anthropic). The AI contributed code (analysis tools, proxy implementation, experiment framework), data analysis, and paper drafting under human direction. The human contributed research design, hypothesis formulation, editorial judgment, and the observation that the waste pattern generalizes beyond Claude Code.

The experiment framework (pichay) was itself built by the system being studied—Claude Code running through its own proxy—making this work partially self-referential. We consider this a feature, not a bug: the system's ability to measure and optimize its own context usage is evidence that the interventions are practical.

All instruments and data are open source: https://github.com/fsgeek/pichay.

References

deepset Team. Context engineering: The next frontier beyond prompt engineering. https://www.deepset.ai/blog/context-engineering-the-next-frontier-beyond-prompt-engineering, 2026. Accessed March 2026.

Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968. doi: 10.1145/363095.363141.

Peter J. Denning. Virtual memory. *ACM Computing Surveys*, 2(3):153–189, 1970. doi: 10.1145/356571.356573.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. *arXiv preprint arXiv:2510.00615*, 2025.

Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management. In *4th Deep Learning for Code Workshop (DL4C) at NeurIPS 2025*, 2025. arXiv:2508.21433.

Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A survey of context engineering for large language models. *arXiv preprint arXiv:2507.13334*, 2025.

Prithvi Rajasekaran, Ethan Dixon, Carly Ryan, and Jeremy Hadfield. Effective context engineering for AI agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, 2025. Accessed March 2026.

Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. SWE-Pruner: Self-adaptive context pruning for coding agents. *arXiv preprint arXiv:2601.16746*, 2026.

Simon Willison. Context engineering. https://simonwillison.net/2025/jun/27/context-engineering/, 2025. Accessed March 2026.

Philip Zeyliger. Expensively quadratic: the LLM agent cost curve. https://blog.exe.dev/expensively-quadratic, 2026. Accessed March 2026.